

Παράλληλος Προγραμματισμός 2026

1η Προγραμματιστική Εργασία

Νικόλαος Πουλόπουλος

AM: 3230170

E-mail: p3230170@aueb.gr

Το 1ο σετ ασκήσεων αφορά στον παράλληλο προγραμματισμό με το μοντέλο κοινόχρηστου χώρου διευθύνσεων μέσω του προτύπου POSIX threads (pthreads). Ζητείται η παραλληλοποίηση του υπολογισμού αριθμητικής ολοκλήρωσης μιας συνάρτησης $f(x)$ με τον Τραπεζοειδή Τύπο εξετάζοντας διαφορετικούς τρόπους διαμοιρασμού της δουλειάς μεταξύ των νημάτων και συγκρίνοντας την απόδοσή τους με τον σειριακό κώδικα.

Όλες οι μετρήσεις έγιναν στο παρακάτω σύστημα:

Όνομα υπολογιστή	Nikolas
Επεξεργαστής	AMD Ryzen 5 5500
Πλήθος πυρήνων	6
Μεταφραστής	gcc v15.2.0

Η μεταγλώττιση έγινε με τις επιλογές `-std=c++17 -pthread` χωρίς επίπεδα βελτιστοποίησης (`-O1/-O2/-O3`) !

Το πρόβλημα

Στην άσκηση αυτή ζητείται να παραλληλοποιηθεί ο υπολογισμός της αριθμητικής ολοκλήρωσης μιας συνάρτησης $f(x)$ στο διάστημα $[a,b]$ με χρήση του Τραπεζοειδούς κανόνα. Δίνεται ένας σειριακός κώδικας σε C++ που προσεγγίζει το ολοκλήρωμα με n υποδιαιρέσεις.

$a = 0$, $b = 10$ και μεγάλο N , υπολογίζει προσεγγιστικά το ολοκλήρωμα

$$\int_a^b f(x)dx \approx \frac{h}{2} \left[f(a) + f(b) + 2 \sum_{i=1}^{n-1} f(a + ih) \right], h = (b - a)/n$$

Στόχος είναι να υλοποιηθούν διαφορετικές παράλληλες εκδόσεις με χρήση pthreads, οι οποίες να εκτελούν τον ίδιο υπολογισμό με διαφορετικό αριθμό νημάτων και να αξιολογείται η απόδοση.

Συγκεκριμένα υλοποιούνται και συγκρίνονται:

- Στατικός διαμοιρασμός σε συνεχόμενα τμήματα, με χρήση κλειδαριών και χωρίς χρήση κλειδαριών.
- Διαμοιρασμός επαναλήψεων με άλματα.
- Δυναμικός διαμοιρασμός μέσω ουράς εργασιών.

Για κάθε υλοποίηση πραγματοποιούνται μετρήσεις χρόνου εκτέλεσης με στόχο την αξιολόγηση της απόδοσης της κατανομής φορτίου και της επίδρασης του αριθμού των νημάτων

Υλοποιήσεις

Σειριακή υλοποίηση

Ο δοσμένος σειριακός κώδικας υλοποιεί τον Τραπεζοειδή κανόνα για $f(x)=x^2$ στο $[0,10]$ με

$N=10^8$ υποδιαιρέσεις και χρησιμοποιείται ως βάση για τη σύγκριση των παράλληλων εκδόσεων. Τρέχοντας τον κώδικα 5 φορές οι μετρούμενοι χρόνοι εκτέλεσης ήταν

Εκτέλεση	Χρόνος (sec)
1η	0.391210
2η	0.390687
3η	0.391095
4η	0.391601
5η	0.394014
Μέσος	0.391721

Οδηγίες Εκτέλεσης

- Compile: `g++ -std=c++17 .\integration.cpp -o .\integration -pthread`
- Run: `.\integration.exe`

Μέθοδος παραλληλοποίησης (A: Στατικός διαμοιρασμός σε συνεχόμενα τμήματα)

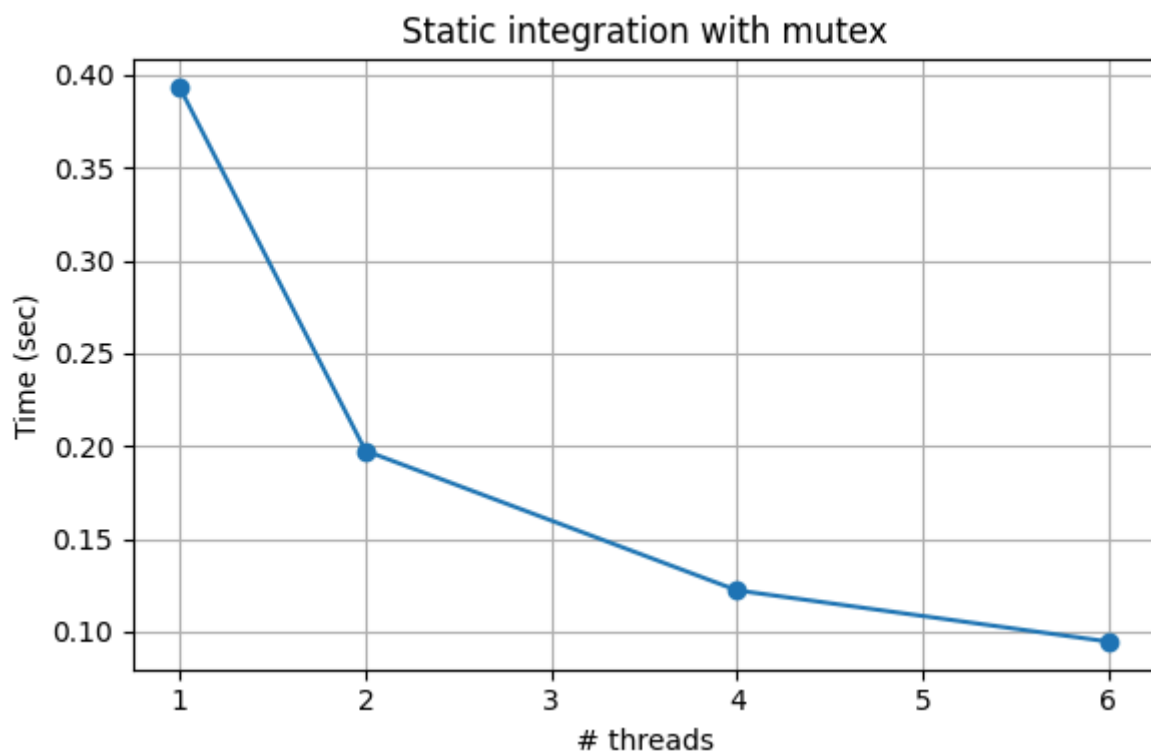
Στην εκδοχή με mutex

- Κάθε νήμα υπολογίζει το μερίδιό του από τα τραπεζοειδή (ένα συνεχόμενο υποδιάστημα του $[0, N)$) και αποθηκεύει το αποτέλεσμα σε μια τοπική μεταβλητή `local_sum`. Στη συνέχεια, μπαίνει σε κρίσιμη περιοχή προστατευμένη με `pthread_mutex_lock` ώστε να προσθέσει το `local_sum` σε ένα κοινό `global_sum` και να πραγματοποιήσει προοδευτική εκτύπωση της μερικής ολοκλήρωσης. Ο στατικός διαμοιρασμός εξασφαλίζει ότι κάθε νήμα

λαμβάνει περίπου τον ίδιο αριθμό επαναλήψεων, ενώ ο mutex εγγυάται ορθή πρόσβαση στο κοινό άθροισμα.

- Πίνακας χρόνων (με mutex, $N = 100000000$)

Νήματα	1η εκτέλεση	2η εκτέλεση	3η εκτέλεση	4η εκτέλεση	5η εκτέλεση	Μέσος όρος (sec)
1	0.398968	0.391053	0.390219	0.397075	0.388534	0.393170
2	0.197857	0.197353	0.196694	0.198551	0.197005	0.197492
4	0.115158	0.126256	0.132995	0.110370	0.126551	0.122266
6	0.101796	0.0916992	0.102214	0.0869936	0.0906026	0.094661



- Με βάση τα αποτελέσματα παρατηρούμε ότι ο μέσος χρόνος εκτέλεσης μειώνεται αισθητά καθώς αυξάνεται ο αριθμός των νημάτων. Από τα περίπου 0.393 sec με (1 νήμα), ο χρόνος πέφτει στα 0.197 sec (2 νήματα), 0.112 sec (4 νήματα) και 0.094 sec (6 νήματα), δείχνοντας σημαντική επιτάχυνση ειδικά μέχρι τα 4 νήματα.
- Η συμπεριφορά αυτή είναι αναμενόμενη, καθώς ο στατικός διαμοιρασμός σε συνεχόμενα τμήματα μοιράζει ισόποσα τις επαναλήψεις και κάθε επανάληψη έχει παρόμοιο φόρτο υπολογισμών, επομένως τα νήματα μπορούν να αξιοποιούν αποτελεσματικά τους διαθέσιμους πυρήνες. Παρότι χρησιμοποιείται mutex για την ενημέρωση του κοινόχρηστου αποτελέσματος και για την προοδευτική εκτύπωση, το κόστος συγχρονισμού παραμένει σχετικά μικρό σε σχέση με τον συνολικό υπολογισμό για $N=10^8$ οπότε δεν αναιρεί το κέρδος από την παραλληλοποίηση.
- Σε σύγκριση με τον σειριακό μέσο χρόνο 0.3917 sec η εκδοχή με mutex και 6 νήματα 0.0884 sec επιτυγχάνει speedup περίπου 4.1 ($0.3917/0.0946 \approx 4.14$) που είναι σημαντικό αλλά όχι πλήρως γραμμικό λόγω του κόστους συγχρονισμού και του σειριακού τμήματος του κώδικα.

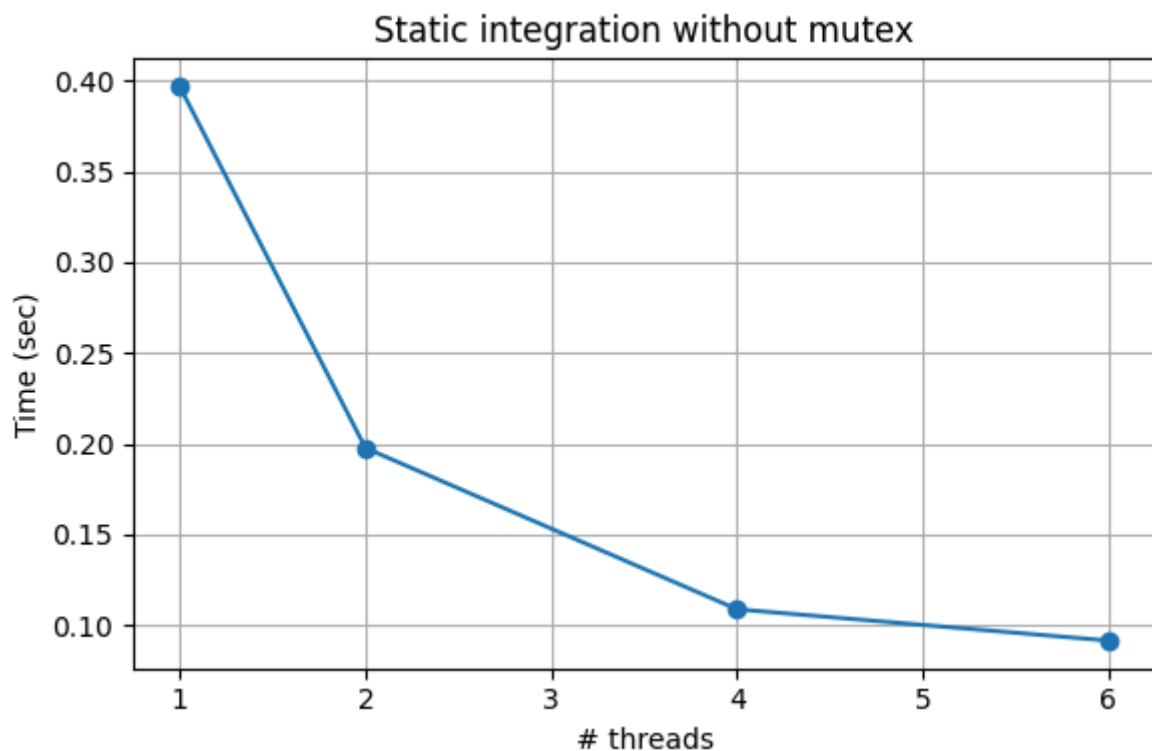
Οδηγίες Εκτέλεσης

- Compile: `g++ -std=c++17 .\static_integration_with_mutex.cpp -o .\static_integration_with_mutex -pthread`
- Run: `.\static_integration_with_mutex.exe 100000000 threads_num`

Στην εκδοχή χωρίς mutex

- Ο διαμοιρασμός των επαναλήψεων παραμένει στατικός, αλλά αντί να ενημερώνει ένα κοινό `global_sum`, κάθε νήμα γράφει το αποτέλεσμα των υπολογισμών του σε μια ξεχωριστή θέση ενός πίνακα `local_sums`. Μετά την ολοκλήρωση όλων των νημάτων, το κύριο νήμα αθροίζει σειριακά τα περιεχόμενα του `local_sums` για να προκύψει η τελική τιμή της ολοκλήρωσης. Έτσι αποφεύγεται πλήρως η χρήση κλειδαριών, μειώνοντας το κόστος συγχρονισμού, με αντάλλαγμα ένα μικρό επιπλέον σειριακό βήμα στο τέλος.
- Πίνακας χρόνων (χωρίς mutex, $N = 100000000$)

Νήματα	1η εκτέλεση	2η εκτέλεση	3η εκτέλεση	4η εκτέλεση	5η εκτέλεση	Μέσος όρος (sec)
1	0.397301	0.391454	0.417784	0.39068	0.386739	0.39679196
2	0.195040	0.201763	0.198791	0.197391	0.195499	0.19769661
4	0.100410	0.114407	0.111573	0.103162	0.114821	0.10887460
6	0.085495	0.115971	0.085837	0.085550	0.085234	0.09161740



- Με βάση τα αποτελέσματα παρατηρούμε ότι ο μέσος χρόνος εκτέλεσης μειώνεται σημαντικά καθώς αυξάνεται ο αριθμός των νημάτων. Από τα 0.396 sec με (1 νήμα), ο χρόνος πέφτει περίπου στα 0.197 sec (2 νήματα), 0.108 sec (4 νήματα) και 0.091 sec (6 νήματα), δηλαδή πετυχαίνουμε ισχυρή επιτάχυνση, ιδιαίτερα μέχρι τα 4 νήματα.
- Η συμπεριφορά αυτή είναι αναμενόμενη, καθώς οι επαναλήψεις του βρόχου ολοκλήρωσης μοιράζονται ισόποσα και κάθε επανάληψη έχει παρόμοιο φόρτο υπολογισμών. Επιπλέον, η συγκεκριμένη υλοποίηση δεν χρησιμοποιεί mutex για την ενημέρωση του αποτελέσματος, άρα δεν πληρώνει κόστος συγχρονισμού και τα νήματα δουλεύουν πλήρως ανεξάρτητα, συνδυάζοντας τα μερικά αποτελέσματα μόνο στο τέλος με ένα σειριακό άθροισμα.

- Σε σχέση με τον σειριακό κώδικα σειριακό μέσο χρόνο 0.3917 sec η εκδοχή χωρίς mutex με 6 νήματα 0.091 sec δίνει speedup περίπου 4.0 ($0.3917/0.091 \approx 4.30$) κοντά στις άλλες υλοποιήσεις ενώ τα καλύτερα κέρδη εμφανίζονται 4 νήματα όπου η αφαίρεση του lock από τον κύριο loop μειώνει το κόστος συγχρονισμού χωρίς να περιορίζεται ακόμη η παραλληλία από άλλους παράγοντες.

Οδηγίες Εκτέλεσης

- Compile: `g++ -std=c++17 .\static_integration_without_mutex.cpp -o .\static_integration_without_mutex -pthread`
- Run: `.\static_integration_without_mutex.exe 100000000 threads_num`

Μέθοδος παραλληλοποίησης(B: Δρομολόγηση επαναλήψεων με άλματα)

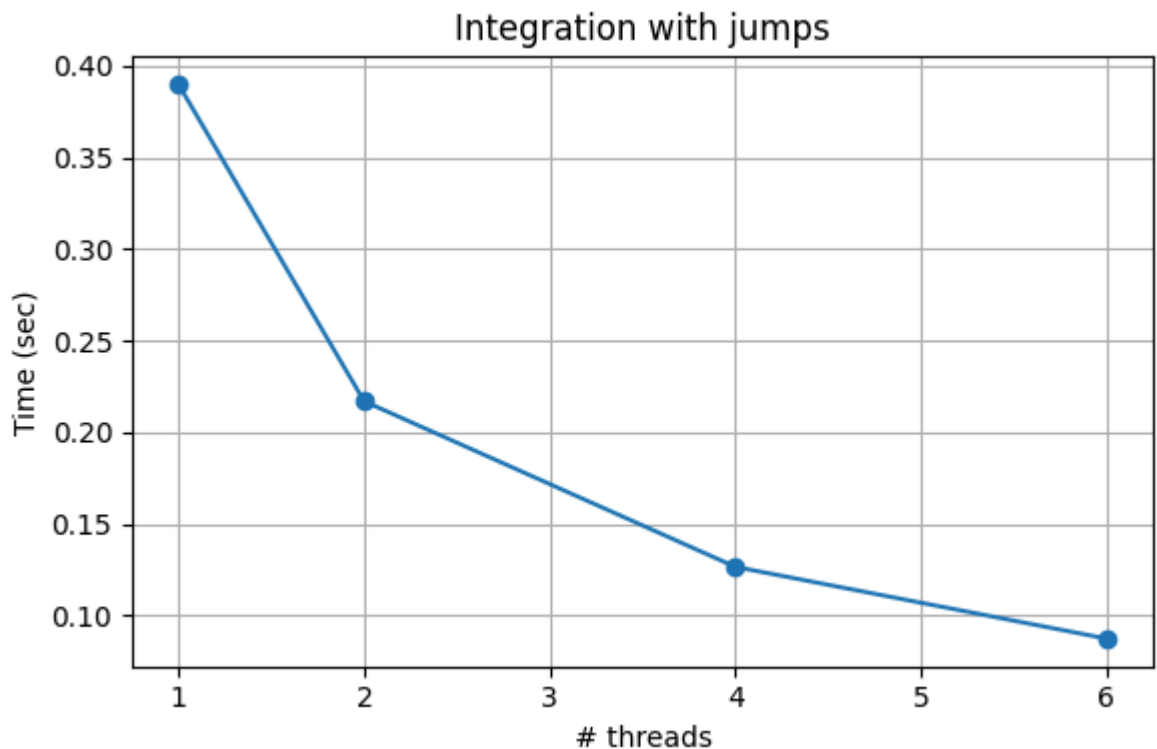
Στην εκδοχή με $f(x) = x^2$

- Στη δεύτερη παράλληλη υλοποίηση ζητείται η δρομολόγηση των επαναλήψεων του βρόχου ολοκλήρωσης με άλματα αντί για συνεχόμενα τμήματα. Στόχος είναι τα νήματα να μοιράζονται ομοιόμορφα τις επαναλήψεις πάνω σε ολόκληρο το διάστημα $[0, N)$ ώστε να παρατηρηθεί η επίδραση αυτού του τρόπου διαμοιρασμού στην απόδοση. Η αριθμητική ολοκλήρωση παραμένει η ίδια (τραπεζοειδής κανόνας για $f(x) = x^2$ στο $[0, 10]$), αλλά αλλάζει ο τρόπος που χωρίζεται ο βρόχος στις επαναλήψεις μεταξύ των νημάτων.
- Χρησιμοποιείται η ίδια βασική δομή κώδικα με τις στατικές υλοποιήσεις, αλλά ο βρόχος ολοκλήρωσης τροποποιείται ώστε κάθε νήμα να εκτελεί επαναλήψεις με βήμα ίσο με το πλήθος των νημάτων. Συγκεκριμένα για νήμα με ταυτότητα `threadid` και συνολικό πλήθος νημάτων `thread_count` το εύρος επαναλήψεων ορίζεται ως:
 - `for (long i = threadid; i < N; i += thread_count) {`
 - `double x1 = a + i * h;`
 - `double x2 = a + (i + 1) * h;`
 - `local_sum += (f(x1) + f(x2)) * h / 2.0;`
 - `}`
- Όπως και στην υλοποίηση με mutex του στατικού διαμοιρασμού, κάθε νήμα υπολογίζει πρώτα ένα τοπικό άθροισμα `local_sum` στο δικό του εύρος

επαναλήψεων και στη συνέχεια ενημερώνει το κοινό `global_sum` μέσα από μία κρίσιμη περιοχή:

- `pthread_mutex_lock(&sum_mutex);`
 - `global_sum += local_sum;`
 - `pthread_mutex_unlock(&sum_mutex);`
- Η προσέγγιση αυτή κρατά τον συγχρονισμό στο ελάχιστο (μία κλήση `lock/unlock` ανά νήμα) και εξασφαλίζει ορθότητα του τελικού αποτελέσματος, καθώς όλα τα `thread-local` αθροίσματα συνδυάζονται με ασφάλεια σε ένα κοινό άθροισμα ολοκλήρωσης
- Πίνακας χρόνων (χωρίς `mutex`, $N = 100000000$)

Νήματα	1η εκτέλεση	2η εκτέλεση	3η εκτέλεση	4η εκτέλεση	5η εκτέλεση	Μέσος όρος (sec)
1	0.387917	0.391464	0.390371	0.389029	0.390578	0.3898718
2	0.209785	0.255852	0.207048	0.205589	0.206269	0.2169086
4	0.136725	0.123469	0.131354	0.122003	0.119040	0.1265182
6	0.0931242	0.0794383	0.0909372	0.0809939	0.0923245	0.0873636



- Με βάση τα αποτελέσματα παρατηρούμε ότι ο μέσος χρόνος εκτέλεσης μειώνεται καθώς αυξάνεται ο αριθμός των νημάτων και στην υλοποίηση με άλματα. Από περίπου 0.389 sec με (1 νήμα) ο χρόνος πέφτει σε περίπου 0.216 sec με 2 νήματα, 0.126 sec με 4 νήματα και 0.087 sec με 6 νήματα δείχνοντας σαφή επιτάχυνση, ιδιαίτερα μέχρι τα 4 νήματα.
- Η συμπεριφορά αυτή είναι αναμενόμενη καθώς ο διαμοιρασμός των επαναλήψεων εξασφαλίζει ότι κάθε νήμα επεξεργάζεται διάσπαρτα υποδιαστήματα σε όλο το $[a, b]$, με παρόμοιο υπολογιστικό κόστος ανά επανάληψη, άρα ο φόρτος εργασίας κατανέμεται σχετικά ομοιόμορφα.
- Ο συγχρονισμός μέσω mutex γίνεται μόνο μία φορά ανά νήμα στο τέλος του βρόχου, και όχι σε κάθε επανάληψη με αποτέλεσμα το κόστος κλειδώματος/ξεκλειδώματος να είναι μικρό. Μικρές διακυμάνσεις στους χρόνους (όπως στην δεύτερη εκτέλεση για 2 νήματα) μπορούν να αποδοθούν σε εξωτερικούς παράγοντες του λειτουργικού συστήματος και της χρονοδρομολόγησης των νημάτων αλλά η γενική τάση παραμένει φθίνουσα με την αύξηση του αριθμού των νημάτων.
- Η υλοποίηση με άλματα για $f(x)=x^2$ με μέσο χρόνο περίπου 0.087 sec στα 6 νήματα, προσφέρει speedup περίπου 4.5 ($0.3917/0.087 \approx 4.50$) σε σχέση με τον σειριακό πολύ κοντά στις στατικές εκδοχές κάτι που είναι αναμενόμενο αφού το φορτίο ανά επανάληψη είναι ομοιόμορφο και οι διαφορές στον τρόπο διαμοιρασμού δεν επηρεάζουν ουσιαστικά την ισοκατανομή του έργου

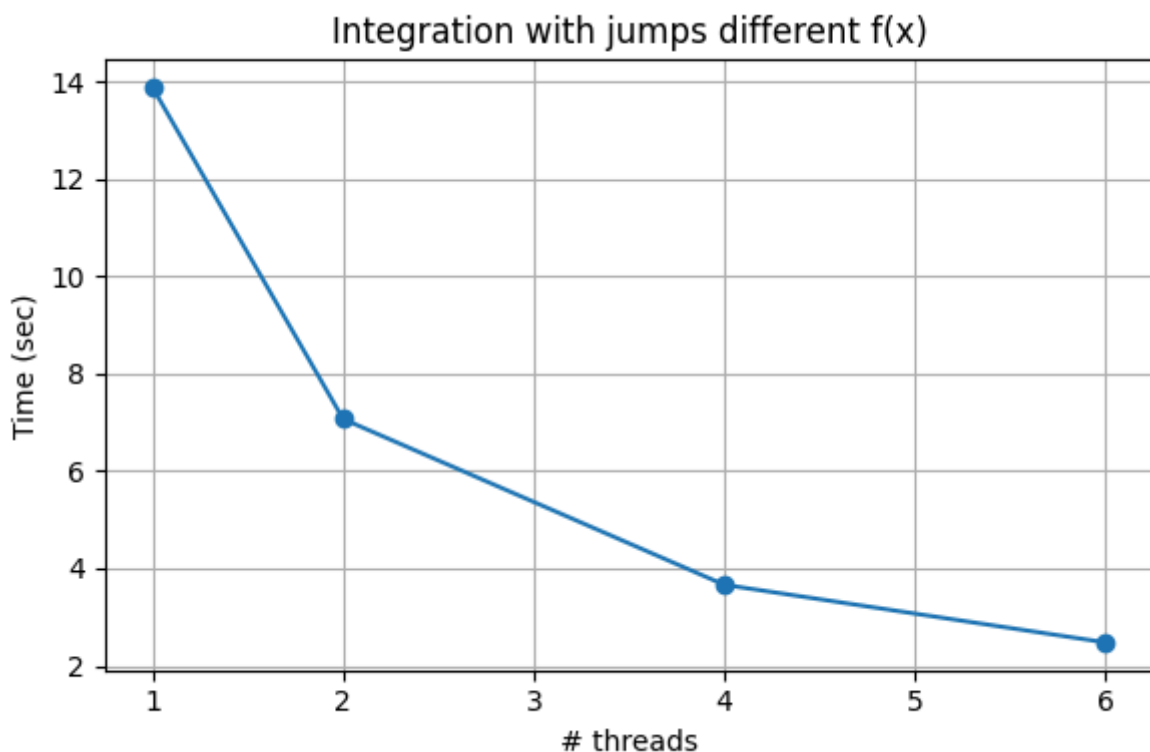
Οδηγίες Εκτέλεσης

- Compile: `g++ -std=c++17 .\integration_with_jumps.cpp -o .\integration_with_jumps -pthread`
- Run: `.\integration_with_jumps.exe 100000000 threads_num`

Στην εκδοχή με τροποποιημένο $f(x) = x^2$

- Πίνακας χρόνων (χωρίς mutex, $N = 100000000$)

Νήματα	1η εκτέλεση	2η εκτέλεση	3η εκτέλεση	4η εκτέλεση	5η εκτέλεση	Μέσος όρος (sec)
1	13.5445	13.9168	14.1470	13.9108	13.8647	13.87676
2	7.07629	7.08296	6.99480	7.09049	7.09492	7.06789
4	3.64066	3.62727	3.61249	3.66687	3.76783	3.66302
6	2.47681	2.48857	2.47335	2.47815	2.48022	2.47942



- Με την τροποποιημένη $f(x)$ ο χρόνος εκτέλεσης αυξάνεται δραματικά σε σχέση με την απλή $f(x)=x^2$ καθώς για κάθε σημείο με $x>5$ εκτελείται πρόσθετος υπολογισμός μέσα σε βρόχο 50 επαναλήψεων κάνοντας κάθε επανάληψη

πολύ πιο βαριά υπολογιστικά. Παρόλα αυτά η υλοποίηση με άλματα εξακολουθεί να κλιμακώνεται ικανοποιητικά με τον αριθμό των νημάτων: ο μέσος χρόνος πέφτει από περίπου 13.87 sec με (1 νήμα) σε 7.06 sec με (2 νήματα), 3.66 sec με (4 νήματα) και 2.47 sec με (6 νήματα) επιτυγχάνοντας ξεκάθαρη επιτάχυνση.

- Η συμπεριφορά αυτή είναι αναμενόμενη γιατί ο διαμοιρασμός με άλματα αναγκάζει κάθε νήμα να επεξεργάζεται σημεία τόσο στο ελαφρύ τμήμα του διαστήματος ($x \leq 5$) όσο και στο βαρύ ($x > 5$) άρα ο πρόσθετος φόρτος μοιράζεται σχετικά ομοιόμορφα ανάμεσα στα νήματα.
- Σε αντίθεση με έναν απλό στατικό διαμοιρασμό σε συνεχόμενα τμήματα όπου ένα νήμα μπορεί να πέσει σχεδόν αποκλειστικά στο ακριβό μέρος του διαστήματος και να καθυστερεί τα υπόλοιπα η δρομολόγηση με άλματα μειώνει τον κίνδυνο κατανομής φορτίου και διατηρεί καλή απόδοση ακόμη και όταν το κόστος ανά επανάληψη είναι σημαντικά ανομοιόμορφο.

Οδηγίες Εκτέλεσης

- Compile: `g++ -std=c++17 .integration_with_jumps_different_fx.cpp -o .integration_with_jumps_different_fx -pthread`
- Run: `.integration_with_jumps_different_fx.exe 100000000 threads_num`

Μέθοδος παραλληλοποίησης(Γ: Δυναμικό διαμοιρασμό μέσω ουράς εργασιών)

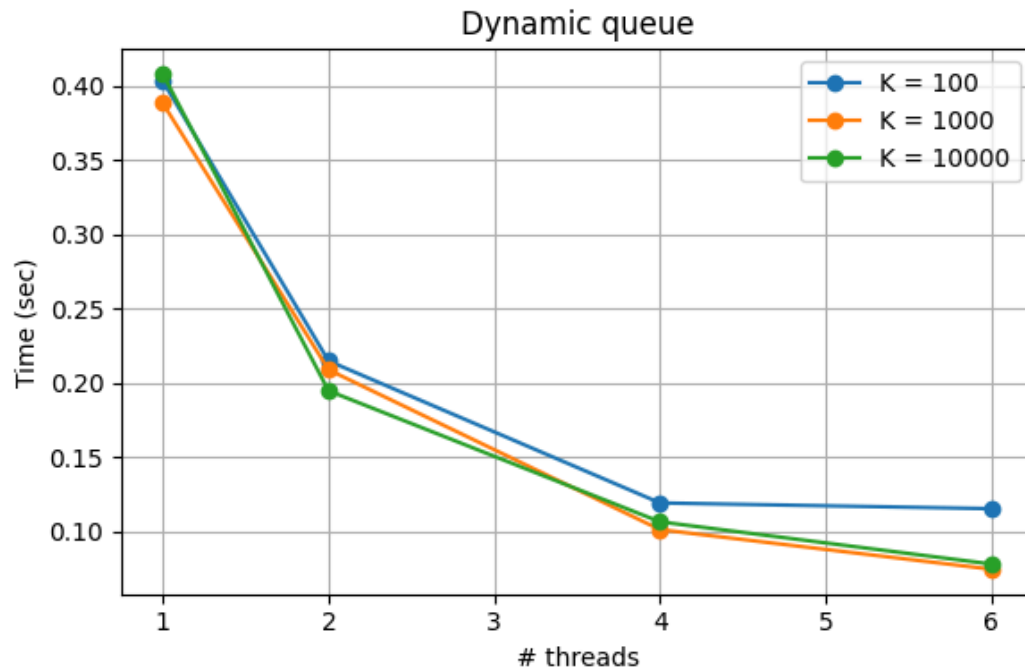
Στην εκδοχή με $f(x) = x^2$

- Στην τρίτη υλοποίηση ο διαμοιρασμός της δουλειάς γίνεται δυναμικά μέσω μιας κοινόχρηστης ουράς εργασιών (task queue). Κάθε εργασία (task) αντιστοιχεί σε ένα συνεχόμενο μπλοκ k επαναλήψεων του βρόχου ολοκλήρωσης
- Το συνολικό πλήθος tasks υπολογίζεται ως $NTASK = \lceil N/K \rceil$ ενώ μια global μεταβλητή taskid δείχνει το επόμενο διαθέσιμο task στην ουρά.
- Κάθε νήμα εκτελεί επαναληπτικά τον βρόχο με κλείδωμα στον mutex tlock αυξάνει το taskid και παίρνει τον αριθμό του επόμενου task αν αυτός είναι μεγαλύτερος ή ίσος από NTASK το νήμα τερματίζει διαφορετικά υπολογίζει τοπικά την ολοκλήρωση στο εύρος [start, end] που αντιστοιχεί σε αυτό το task.

Το άθροισμα για το task συσσωρεύεται σε μια τοπική μεταβλητή `local_sum` και στη συνέχεια με χρήση του `mutex sum_mutex` προστίθεται στο κοινό `global_sum`. Με αυτόν τον τρόπο όποιο νήμα τελειώνει ένα task μπορεί να πάρει αμέσως το επόμενο εξασφαλίζοντας καλό load balancing ακόμη και αν ο όγκος των tasks δεν είναι ομοιόμορφος

- Πίνακας χρόνων (χωρίς mutex, N = 100000000)

Νήματα	K	Μέσος χρόνος μετά από 5 φορές (sec)
1	100	0.4030
1	1000	0.3881
1	10000	0.4077
2	100	0.2146
2	1000	0.2087
2	10000	0.1945
4	100	0.1191
4	1000	0.1011
4	10000	0.1063
6	100	0.1152
6	1000	0.0744
6	10000	0.0779



- Με βάση τα αποτελέσματα παρατηρούμε ότι ο δυναμικός διαμοιρασμός μέσω ουράς εργασιών κλιμακώνεται πολύ καλά με τον αριθμό των νημάτων. Για παράδειγμα με $K=1000$ ο μέσος χρόνος εκτέλεσης μειώνεται από περίπου 0.388 sec με 1 νήμα σε 0.209 sec με 2 νήματα, 0.101 sec με 4 νήματα και 0.074 sec με 6 νήματα επιτυγχάνοντας σημαντική επιτάχυνση και καλή αξιοποίηση των διαθέσιμων πυρήνων.
- Η συμπεριφορά αυτή είναι αναμενόμενη καθώς κάθε νήμα όταν ολοκληρώνει ένα task ζητά άμεσα το επόμενο από την κοινή ουρά έτσι αν κάποια tasks τύχουν πιο βαριά από άλλα ή αν το λειτουργικό σύστημα καθυστερήσει προσωρινά κάποιο νήμα τα υπόλοιπα μπορούν να αναλάβουν περισσότερες εργασίες μειώνοντας το φορτίο.
- Όσον αφορά την επιλογή του μεγέθους task K παρατηρούμε ότι για ένα νήμα η επίδραση είναι μικρή αφού όλα τα tasks εκτελούνται σειριακά και το μόνο που αλλάζει είναι το πόσο συχνά ενημερώνεται ο taskid. Για περισσότερα νήματα και πολύ μικρό K (π.χ. 100) σημαίνει περισσότερα tasks και καλύτερο δυναμικό load balancing αλλά αυξημένο κόστος συγχρονισμού στην ουρά (πολλά lock/unlock στον tlock). Αντίθετα πολύ μεγάλο K (π.χ. 10000) μειώνει τον αριθμό locks αλλά περιορίζει την ευελιξία της ουράς αφού κάθε task είναι βαρύ και αν κάποιο νήμα καθυστερήσει σε ένα τέτοιο task οι υπόλοιποι μπορεί να μείνουν χωρίς δουλειά. Στα συγκεκριμένα πειράματα για 4 και 6 νήματα οι καλύτεροι χρόνοι επιτυγχάνονται γύρω από $K = 1000$, που φαίνεται να προσφέρει καλή ισορροπία μεταξύ κόστους συγχρονισμού και ποιότητας load balancing.
- Για τον δυναμικό διαμοιρασμό με ουρά εργασιών και $K=1000$ ο μέσος χρόνος με 6 νήματα 0.0744 sec αντιστοιχεί σε speedup περίπου 5.3 ($0.3911/0.0744$)

που είναι το καλύτερο από όλα τα σενάρια με απλό $f(x)=x^2$ χάρη στον πολύ αποδοτικό διαμοιρασμό φόρτου και στο γεγονός ότι ο συγχρονισμός περιορίζεται σε επίπεδο task και όχι σε κάθε επανάληψη.

Οδηγίες Εκτέλεσης

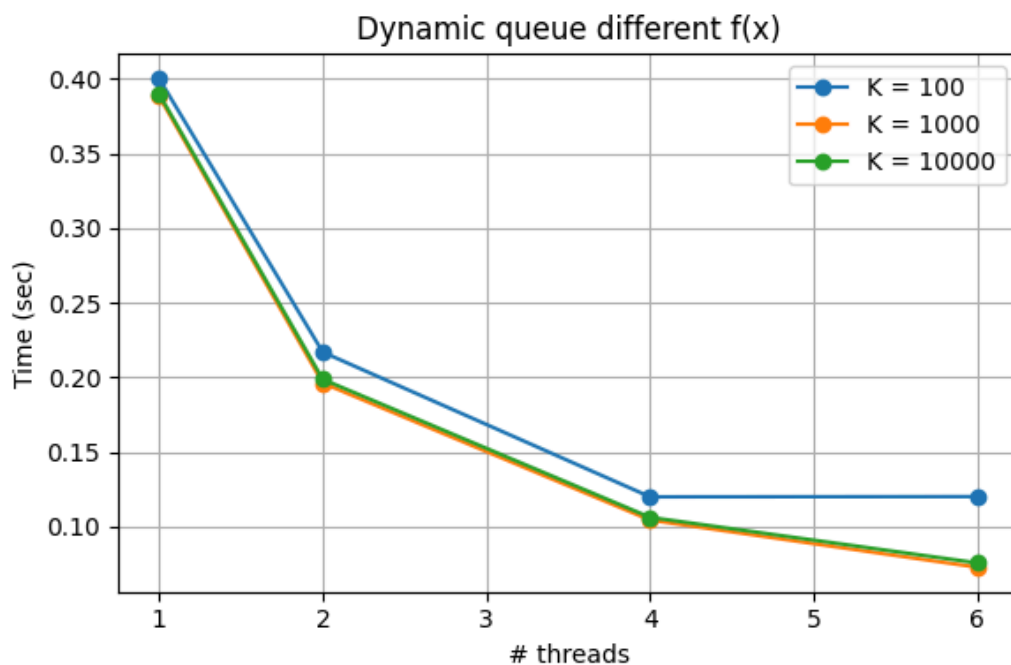
- Compile: `g++ -std=c++17 .integration_dynamic_queue.cpp -o .integration_dynamic_queue -pthread`
- Run: `.integration_dynamic_queue.exe 100000000 threads_num megethos_task`

Στην εκδοχή με τροποποιημένο $f(x) = x^2$

- Για να μελετηθεί η συμπεριφορά της δυναμικής ουράς εργασιών σε συνθήκες άνισου φόρτου τροποποιήθηκε η συνάρτηση $f(x)$ έτσι ώστε το υπολογιστικό κόστος να μην είναι ομοιόμορφο σε όλο το διάστημα ολοκλήρωσης. Συγκεκριμένα για $x \leq 5$ διατηρείται η μορφή $f(x)=x^2$ ενώ για $x > 5$ προστίθεται ένας τεχνητός υπολογισμός μέσα σε έναν βρόχο 50 επαναλήψεων αυξάνοντας σημαντικά τον χρόνο εκτέλεσης σε αυτό το τμήμα του διαστήματος. Με τον τρόπο αυτό δημιουργούνται βαριά tasks που αντιστοιχούν σε υποδιαστήματα όπου τα σημεία βρίσκονται κυρίως στο $[5,10]$ ενώ τα υπόλοιπα tasks παραμένουν ελαφρύτερα.
- Πίνακας χρόνων (χωρίς mutex, $N = 100000000$)

Νήματα	K	Μέσος χρόνος μετά από 5 φορές (sec)
1	100	0.4001
1	1000	0.3883
1	10000	0.3898
2	100	0.2170
2	1000	0.1963

2	10000	0.1988
4	100	0.1201
4	1000	0.1045
4	10000	0.1063
6	100	0.1202
6	1000	0.0729
6	10000	0.0758



- Με την τροποποιημένη $f(x)$ οι συνολικοί χρόνοι εκτέλεσης αυξάνονται σε σχέση με την απλή x^2 καθώς ένα μέρος των επαναλήψεων (εκεί όπου $x > 5$) γίνεται υπολογιστικά πιο ακριβό. Παρόλα αυτά η υλοποίηση με dynamic queue συνεχίζει να κλιμακώνεται καλά με τον αριθμό των νημάτων. Για παράδειγμα με $K=1000$ ο μέσος χρόνος πέφτει από περίπου 0.388 sec (1 νήμα) σε 0.196 sec (2 νήματα), 0.105 sec (4 νήματα) και 0.073 sec (6 νήματα, στρογγυλοποιημένα) επιτυγχάνοντας σημαντική επιτάχυνση παρά την αύξηση του φόρτου.
- Η συμπεριφορά αυτή είναι αναμενόμενη διότι ο δυναμικός διαμοιρασμός μέσω ουράς επιτρέπει στα νήματα να παίρνουν διαδοχικά tasks μέχρι να εξαντληθούν όλα ανεξάρτητα από το πόσο βαρύ είναι το καθένα. Αν κάποιο task τύχει να περιέχει πολλές τιμές στο $[5, 10]$ και απαιτεί περισσότερο χρόνο

τα υπόλοιπα νήματα μπορούν να αναλάβουν επιπλέον tasks από την ουρά αντί να μένουν ανενεργά, κάτι που βελτιώνει την ισοκατανομή του φόρτου και μειώνει τον συνολικό χρόνο εκτέλεσης.

- Όσον αφορά το μέγεθος του task K , τα αποτελέσματα δείχνουν ότι με 1 νήμα οι διαφορές είναι πολύ μικρές όπως αναμενόταν αφού όλες οι εργασίες εκτελούνται σειριακά και το μόνο που αλλάζει είναι η συχνότητα πρόσβασης στον taskid. Για περισσότερα νήματα πολύ μικρό K (π.χ. 100) αυξάνει τον αριθμό των tasks και το κόστος συγχρονισμού στην ουρά ενώ πολύ μεγάλο K (π.χ. 10000) μειώνει τα locks αλλά μπορεί να περιορίσει ελαφρά την ευελιξία διαμοιρασμού. Στα πειράματα οι καλύτεροι χρόνοι για 4 και 6 νήματα προκύπτουν ξανά γύρω από $K=1000$ που αποτελεί έναν καλό συμβιβασμό ανάμεσα σε κόστος συγχρονισμού και αποδοτικό load balancing ακόμη και με ανισοβαρή φόρτο εργασίας.

Οδηγίες Εκτέλεσης

- Compile: `g++ -std=c++17 .\integration_dynamic_queue_different_fx.cpp -o .\integration_dynamic_queue_different_fx -pthread`
- Run: `.\integration_dynamic_queue_different_fx.exe 100000000 threads_num megethos_task`